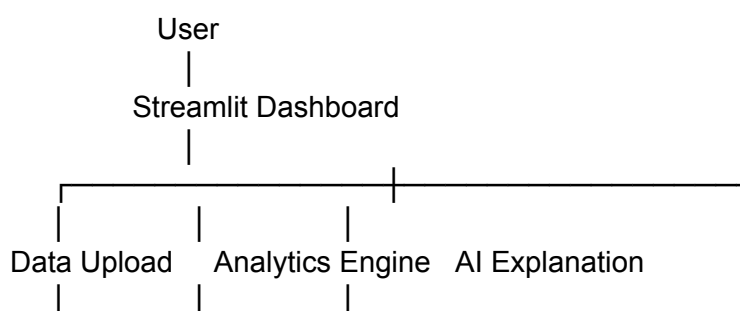
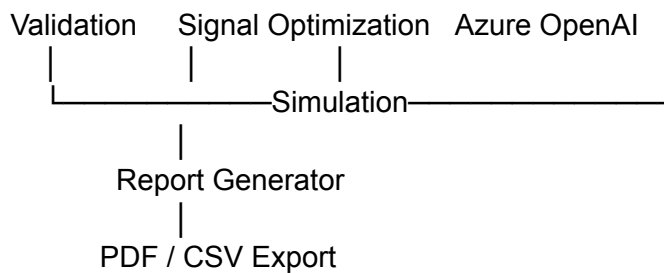


CHAPTER 1 — SYSTEM OVERVIEW

- **System purpose:** MetroFlow AI is an offline municipal traffic analytics and signal timing optimization platform designed to transform historical vehicle count data and road metadata into optimized, explainable, and safe signal timing plans.
- **Functional requirements:** Ingest historical 15-minute traffic count CSVs and road geometry metadata, validate and sanitize input data, perform temporal cluster peak window detection and Volume-to-Capacity (V/C) ratio bottleneck identification, compute Webster-based optimal green splits and cycle lengths, run M/M/1 queuing "before vs. after" simulations, generate Azure OpenAI-powered plain-language engineering rationales, and export NEMA-compliant controller CSVs and PDF executive reports.
- **Non-functional requirements:** Complete corridor optimization calculations in under 5 seconds; return LLM narratives in under 3 seconds; maintain a queuing model variance within $\pm 10\%$ of Highway Capacity Manual (HCM) standards under stable flow; enforce TLS 1.3 encryption in transit and AES-256 at rest; achieve 99.9% uptime availability for the web dashboard.
- **Product workflow:** Users authenticate via secure municipal SSO, upload traffic count CSVs and define intersection metadata, review automated peak congestion and bottleneck heatmaps, execute the offline optimization engine to adjust green splits, evaluate side-by-side queue and delay simulation reductions with stress-test sliders, inspect AI engineering narratives, and download field-ready controller files.
- **High-level architecture:** A modular web application featuring a Streamlit/web dashboard frontend, a Python FastAPI backend orchestrating data validation, analytics, deterministic Webster optimization, safety validation, queuing simulation, and Azure OpenAI integration.
- **Design principles:** Engineering credibility through zero "black box" decisions; extreme simplicity for a 5-minute analyst workflow; safety primacy via hardcoded statutory bounds; explicit simulation assumptions with zero guaranteed performance claims.
- **Assumptions:** Historical counts reflect underlying demand patterns; intersections utilize standard NEMA phasing sequences; driver behavior adheres to baseline Highway Capacity Manual constants unless modified.
- **Constraints:** Strictly offline decision support with zero direct actuation of live traffic controllers; all recommendations serve as advisory deliverables requiring human engineering sign-off.

CHAPTER 2 — SYSTEM ARCHITECTURE





Module Explanations

- **Streamlit Dashboard:** The interactive web frontend providing workspace navigation, drag-and-drop file upload zones, GIS heatmap visualizations, optimization parameter controls, and export triggers.
- **Data Upload & Validation Module:** Parses CSV files, checks timestamps and data types, flags missing or corrupted records, and applies anomaly filters before analysis.
- **Analytics Engine:** Executes rolling 1-hour temporal cluster analysis, computes Volume-to-Capacity (V/C) ratios, and assigns Highway Capacity Manual (HCM) Level of Service (LOS) grades.
- **Signal Optimization Engine:** Applies deterministic Webster formulas to calculate optimal cycle lengths, distribute green splits proportionally, and determine green-wave coordination offsets.
- **Safety Validator:** Intercepts optimization outputs to hardcode and enforce statutory minimum pedestrian walk times and clearance intervals.
- **Simulation Engine:** Utilizes M/M/1 and Webster queuing mathematics to model before-versus-after delay, queue dissipation, and volume-surge stress testing.
- **AI Explanation Module:** Sends structured JSON payload summaries of simulation metrics to Azure OpenAI to produce transparent, plain-language engineering rationales without performing calculations.
- **Export Module:** Compiles NEMA-compliant controller CSV timing sheets and branded executive PDF summary reports.

CHAPTER 3 — MODULE BREAKDOWN

Frontend

- **Purpose:** Provide an intuitive, low-barrier web interface for traffic engineers and municipal consultants.
- **Responsibilities:** Render multi-screen navigation, interactive GIS bottleneck maps, configuration sliders, and real-time comparative charts.
- **Inputs:** User interactions, file uploads, slider values.
- **Outputs:** Formatted JSON payloads sent to backend APIs.
- **Dependencies:** Streamlit / React framework components, charting libraries.

Upload Module

- **Purpose:** Ingest and secure raw municipal traffic logs and metadata.
- **Responsibilities:** Handle file drag-and-drop, parse multi-intersection CSV structures, and register approach metadata.
- **Inputs:** Raw CSV traffic files, user-defined roadway geometry.

- **Outputs:** Cleaned tabular datasets, parsing status logs.
- **Dependencies:** Pandas, NumPy.

Validation Module

- **Purpose:** Ensure input data integrity and prevent corrupted records from corrupting models.
- **Responsibilities:** Check for missing timestamps, negative volumes, duplicate entries, and turning ratios that fail to sum to 1.0.
- **Inputs:** Raw parsed CSV data.
- **Outputs:** Sanitized dataset, warning/error notification list.
- **Dependencies:** Upload module output.

Traffic Analytics

- **Purpose:** Identify temporal congestion patterns and spatial bottlenecks across arterial corridors.
- **Responsibilities:** Execute rolling 1-hour sum windows, calculate V/C ratios, and assign HCM LOS grades.
- **Inputs:** Sanitized 15-minute volume logs, road metadata.
- **Outputs:** Peak time windows, bottleneck intersection rankings, V/C heatmaps.
- **Dependencies:** Validation module.

Peak Detection

- **Purpose:** Isolate primary morning, midday, and evening rush hour windows.
- **Responsibilities:** Cluster continuous traffic volume data across 7-day intervals to pinpoint peak demand blocks.
- **Inputs:** Cleaned 15-minute interval volumes.
- **Outputs:** Peak window start/end timestamps.
- **Dependencies:** Traffic analytics module.

Bottleneck Detection

- **Purpose:** Highlight critical network failures restricting overall corridor flow.
- **Responsibilities:** Compute approach capacities using lane counts and saturation flows, flagging any V/C ratio > 1.0 as LOS F.
- **Inputs:** Hourly flow rates, lane geometry, saturation flow constants.
- **Outputs:** Ranked bottleneck node list.
- **Dependencies:** Peak detection module.

Signal Optimization

- **Purpose:** Generate mathematically sound, offline green split and cycle length recommendations.
- **Responsibilities:** Execute Webster's formula to balance phase allocations proportionally to critical lane volumes.
- **Inputs:** Critical lane volumes, clearance lost times, user-defined cycle bounds.
- **Outputs:** Optimized cycle length, phase green split allocations, coordination offsets.
- **Dependencies:** Bottleneck detection module.

Simulation Engine

- **Purpose:** Model operational performance improvements before field deployment.
- **Responsibilities:** Apply M/M/1 and Webster queuing equations to calculate control delay, queue lengths, and volume surge stress responses.
- **Inputs:** Baseline timings, recommended timings, demand multiplier sliders.
- **Outputs:** Side-by-side metric comparison data, fuel/CO2 estimates.
- **Dependencies:** Signal optimization module.

AI Explanation

- **Purpose:** Bridge complex mathematical output with human-understandable engineering justification.
- **Responsibilities:** Format simulation results into structured JSON and query Azure OpenAI for narrative summaries.
- **Inputs:** Before/after structured metric JSON payloads.
- **Outputs:** Markdown-formatted executive narrative strings.
- **Dependencies:** Simulation engine.

Export Module

- **Purpose:** Package final recommendations into deliverables suitable for municipal review and technician programming.
- **Responsibilities:** Generate downloadable NEMA-compliant CSV timing sheets and executive PDF reports.
- **Inputs:** Optimized phase parameters, AI narratives, project metadata.
- **Outputs:** `.csv` and `.pdf` files.
- **Dependencies:** Signal optimization, AI explanation module.

Utilities

- **Purpose:** Provide shared helper functions across backend modules.
- **Responsibilities:** Handle unit conversions, mathematical rounding, and timestamp formatting.
- **Inputs:** Raw scalar or vector variables.
- **Outputs:** Standardized variables.
- **Dependencies:** None.

Configuration

- **Purpose:** Centralize system constants, environment variables, and default safety parameters.
- **Responsibilities:** Load API keys, default saturation flow rates (1900 veh/hr/lane), and pedestrian walk speeds (3.5 ft/sec).
- **Inputs:** `.env` file, system defaults.
- **Outputs:** Configuration dictionary/object.
- **Dependencies:** None.

CHAPTER 4 — PROJECT STRUCTURE

- frontend/
- backend/
- analytics/
- optimization/
- simulation/
- ai/
- utils/
- config/
- datasets/
- tests/
- docs/

Folder Purpose Explanations

- **frontend/**: Contains the Streamlit multi-page web application UI, custom CSS styles, and component layouts.
- **backend/**: Houses the FastAPI application routes, middleware, and core orchestration controllers.
- **analytics/**: Manages data ingestion parsers, data cleaning routines, peak window clustering, and V/C ratio calculators.
- **optimization/**: Contains deterministic Webster cycle length calculators, green split distribution logic, and safety guardrail validators.
- **simulation/**: Implements M/M/1 queuing models, before/after delay calculations, and what-if stress-test multipliers.
- **ai/**: Manages Azure OpenAI API integrations, prompt generation templates, and structured JSON payload formatters.
- **utils/**: Shared helper scripts for logging, error handling, and mathematical formatting.
- **config/**: Environment configuration templates, system constants, and default traffic engineering parameters.
- **datasets/**: Sample CSV files containing 7-day multi-intersection traffic counts, road geometry, and incident logs for MVP testing.
- **tests/**: Unit and integration test suites covering data validation, queuing math accuracy, and API endpoints.
- **docs/**: Technical specifications, API documentation, and user guides.

CHAPTER 5 — DATABASE DESIGN

Traffic Counts Table

- **Columns**: `count_id` (UUID), `project_id` (UUID), `intersection_id` (VARCHAR), `timestamp` (DATETIME), `approach_direction` (VARCHAR), `volume` (INTEGER).
- **Primary Key**: `count_id`.
- **Foreign Keys**: `project_id` references `projects(project_id)`, `intersection_id` references `road_metadata(intersection_id)`.
- **Relationships**: Many-to-one with Projects; Many-to-one with Road Metadata.
- **Indexes**: `idx_count_lookup` on (`project_id`, `intersection_id`, `timestamp`).
- **Validation Rules**: `volume` ≥ 0 , `timestamp` must follow YYYY-MM-DD HH:MM:SS format.

Road Metadata Table

- **Columns:** `metadata_id` (UUID), `project_id` (UUID), `intersection_id` (VARCHAR), `approach_direction` (VARCHAR), `lanes` (INTEGER), `speed_limit` (INTEGER), `storage_bay_length` (FLOAT).
- **Primary Key:** `metadata_id`.
- **Foreign Keys:** `project_id` references `projects(project_id)`.
- **Relationships:** One-to-many with Traffic Counts and Signal Timings.
- **Indexes:** `idx_meta_intersection` on (`project_id`, `intersection_id`).
- **Validation Rules:** `lanes > 0`, `speed_limit > 0`, `storage_bay_length >= 0`.

Turning Ratios Table

- **Columns:** `ratio_id` (UUID), `intersection_id` (VARCHAR), `approach_direction` (VARCHAR), `left_ratio` (FLOAT), `through_ratio` (FLOAT), `right_ratio` (FLOAT).
- **Primary Key:** `ratio_id`.
- **Foreign Keys:** `intersection_id` references `road_metadata(intersection_id)`.
- **Relationships:** One-to-one with Road Metadata approaches.
- **Indexes:** `idx_turn_intersection` on `intersection_id`.
- **Validation Rules:** `left_ratio + through_ratio + right_ratio == 1.0` (or normalized automatically).

Signal Timings Table

- **Columns:** `timing_id` (UUID), `intersection_id` (VARCHAR), `phase_id` (INTEGER), `current_green` (INTEGER), `min_ped_walk` (INTEGER), `yellow_time` (FLOAT), `optimized_green` (INTEGER).
- **Primary Key:** `timing_id`.
- **Foreign Keys:** `intersection_id` references `road_metadata(intersection_id)`.
- **Relationships:** One-to-many with Road Metadata.
- **Indexes:** `idx_timing_phase` on (`intersection_id`, `phase_id`).
- **Validation Rules:** `current_green >= min_ped_walk`, `yellow_time > 0`, `optimized_green >= min_ped_walk`.

Simulation Results Table

- **Columns:** `simulation_id` (UUID), `project_id` (UUID), `intersection_id` (VARCHAR), `baseline_delay` (FLOAT), `optimized_delay` (FLOAT), `baseline_queue` (FLOAT), `optimized_queue` (FLOAT), `baseline_los` (VARCHAR), `optimized_los` (VARCHAR).
- **Primary Key:** `simulation_id`.
- **Foreign Keys:** `project_id` references `projects(project_id)`.
- **Relationships:** Many-to-one with Projects.
- **Indexes:** `idx_sim_project` on `project_id`.
- **Validation Rules:** Delay and queue values must be ≥ 0 ; LOS must be a valid grade string between `A` and `F`.

Projects Table

- **Columns:** `project_id` (UUID), `user_id` (UUID), `project_name` (VARCHAR), `corridor_name` (VARCHAR), `created_at` (DATETIME), `updated_at` (DATETIME).
- **Primary Key:** `project_id`.

- **Foreign Keys:** None (User account managed via SSO).
- **Relationships:** One-to-many with Traffic Counts, Road Metadata, and Simulation Results.
- **Indexes:** `idx_proj_user` on `user_id`.
- **Validation Rules:** `project_name` must be non-empty.

Reports Table

- **Columns:** `report_id` (UUID), `project_id` (UUID), `ai_narrative` (TEXT), `pdf_report_path` (VARCHAR), `nema_csv_path` (VARCHAR), `generated_at` (DATETIME).
- **Primary Key:** `report_id`.
- **Foreign Keys:** `project_id` references `projects(project_id)`.
- **Relationships:** One-to-one with Projects.
- **Indexes:** `idx_report_proj` on `project_id`.
- **Validation Rules:** File paths must point to valid stored artifact locations.

CHAPTER 6 — DATA FLOW

- **CSV Upload:** User uploads historical 15-minute volume counts and metadata files via the web portal.
- **Validation:** System ingests payloads, checking schema conformity, column headers, and data types.
- **Cleaning:** Null values are interpolated from adjacent intervals, negative numbers floored, and extreme outliers clamped to physical saturation limits.
- **Analytics:** Cleaned volumes are aggregated into hourly flow rates to establish baseline throughput.
- **Peak Detection:** Rolling 1-hour sum clustering isolates AM, midday, and PM peak demand periods.
- **Bottleneck Detection:** Approach capacities are computed using lane geometry and saturation flows, calculating V/C ratios and flagging intersections >1.0 as LOS F bottlenecks.
- **Optimization:** Webster's formula calculates optimal cycle lengths and distributes green time proportionally to critical lane volumes.
- **Simulation:** M/M/1 and Webster queuing formulas model before-versus-after delay reductions and queue dissipation.
- **AI Explanation:** Structured JSON payloads of simulation metrics are transmitted to Azure OpenAI to generate plain-language engineering rationales.
- **Dashboard:** Processed metrics, heatmaps, comparative tables, and interactive stress-test sliders are rendered in the Streamlit UI.
- **Report Export:** Final NEMA CSV timing sheets and branded executive PDF reports are compiled and made available for download.

CHAPTER 7 — API DESIGN

1. Ingest Data

- **Method:** `POST`

- **Endpoint:** `/api/v1/projects/{project_id}/upload`
- **Purpose:** Upload and validate historical traffic count CSVs and road geometry.
- **Input:** Multipart form-data containing CSV file and metadata payload.
- **Output:** JSON confirmation object with row count, validation status, and warning logs.
- **Validation:** File extension must be `.csv`; required headers (`Timestamp`, `Intersection_ID`, `Approach_Direction`, `Volume`) must exist.
- **Errors:** `400 Bad Error` (Invalid format), `422 Unprocessable Entity` (Missing required columns).
- **Example Request:** `POST /api/v1/projects/proj_01/upload` with `traffic_log.csv`.
- **Example Response:** `{"status": "success", "rows_processed": 1440, "warnings": ["Missing volumes interpolated for 08:15 AM"]}`.

2. Run Analytics

- **Method:** `POST`
- **Endpoint:** `/api/v1/projects/{project_id}/analyze`
- **Purpose:** Execute peak window clustering and V/C ratio bottleneck detection.
- **Input:** `{ "corridor_id": "string" }`
- **Output:** JSON payload containing peak windows, V/C ratios per approach, and ranked bottlenecks.
- **Validation:** Project must contain validated traffic and metadata records.
- **Errors:** `404 Not Found` (Project ID invalid), `400 Bad Request` (Data unvalidated).
- **Example Request:** `POST /api/v1/projects/proj_01/analyze`.
- **Example Response:** `{"peak_am": "07:45 - 09:15", "worst_intersection": "INT_04", "vc_ratio": 1.12, "los": "F"}`.

3. Generate Optimization

- **Method:** `POST`
- **Endpoint:** `/api/v1/projects/{project_id}/optimize`
- **Purpose:** Compute Webster cycle lengths, phase green splits, and safety constraints.
- **Input:** `{ "min_cycle": 60, "max_cycle": 180 }`
- **Output:** JSON array of optimized phase green splits and cycle lengths.
- **Validation:** Cycle bounds must be within 60s and 180s.
- **Errors:** `422 Unprocessable Entity` (Cycle bounds violate pedestrian safety minimums).
- **Example Request:** `POST /api/v1/projects/proj_01/optimize` with `{"min_cycle": 90, "max_cycle": 150}`.
- **Example Response:** `{"cycle_length": 120, "phase_splits": {"phase_2": 45, "phase_4": 18}}`.

4. Run Simulation

- **Method:** POST
- **Endpoint:** `/api/v1/projects/{project_id}/simulate`
- **Purpose:** Execute M/M/1 queuing simulations for before-vs-after comparison and volume surges.
- **Input:** `{ "demand_multiplier": 1.2 }`
- **Output:** Comparative metrics for delay, queue length, and LOS.
- **Validation:** Demand multiplier must be between 0.7 and 1.3.
- **Errors:** 400 Bad Request (Invalid multiplier range).
- **Example Request:** POST `/api/v1/projects/proj_01/simulate` with `{ "demand_multiplier": 1.0 }`.
- **Example Response:** `{ "baseline_delay_sec": 68.4, "optimized_delay_sec": 42.1, "delay_reduction_pct": 38.4 }`.

5. Generate AI Rationale

- **Method:** POST
- **Endpoint:** `/api/v1/projects/{project_id}/ai-rationale`
- **Purpose:** Query Azure OpenAI for plain-language engineering explanations based on simulation JSON.
- **Input:** `{ "simulation_results_id": "string" }`
- **Output:** Markdown string containing plain-language executive narrative.
- **Validation:** Simulation results must exist for the project.
- **Errors:** 502 Bad Gateway (Azure OpenAI service timeout).
- **Example Request:** POST `/api/v1/projects/proj_01/ai-rationale`.
- **Example Response:** `{ "narrative": "Green time for Phase 2 was increased by 14 seconds..." }`.

6. Export Reports

- **Method:** GET
- **Endpoint:** `/api/v1/projects/{project_id}/export/{format}`
- **Purpose:** Download NEMA controller CSV timing sheets or executive PDF reports.
- **Input:** Format specifier (`csv` or `pdf`).
- **Output:** File stream attachment.
- **Validation:** Format must be either `csv` or `pdf`.
- **Errors:** 400 Bad Request (Unsupported format).
- **Example Request:** GET `/api/v1/projects/proj_01/export/csv`.
- **Example Response:** Binary file stream (`nema_timing_sheet.csv`).

CHAPTER 8 — ANALYTICS ENGINE

- **Traffic Volume Analysis:** Aggregates 15-minute interval logs into hourly flow rates to establish temporal volume distribution.
- **Peak Detection:** Applies rolling 1-hour sum windows across 7-day datasets to identify AM, midday, and PM peak demand blocks.
- **LOS Calculation:** Evaluates control delay per vehicle to assign Highway Capacity Manual Level of Service grades from A through F.
- **V/C Ratio:** Computes demand-to-capacity ratios (v/c) using approach volume demand divided by lane count multiplied by saturation flow ($\sim 1900 \text{ veh/hr/lane}$) and green ratios.
- **Queue Estimation:** Models vehicle queue accumulation based on arrival rates exceeding discharge capacity during peak intervals.
- **Delay Estimation:** Calculates uniform delay (d_1) and random/overflow delay (d_2) using Webster formulation logic.
- **Ranking:** Orders corridor intersections in descending sequence based on absolute peak approach V/C ratios.
- **Output Objects:** Returns structured JSON dictionaries containing peak start/end times, V/C ratio arrays, LOS grades, and ranked bottleneck node IDs.
- **Dependencies:** NumPy, Pandas, SciPy.

CHAPTER 9 — OPTIMIZATION ENGINE

- **Inputs:** Critical lane volumes (v_c), clearance lost times (L), minimum pedestrian crossing times, and user-defined cycle bounds (60 s to 180 s).
- **Calculations:** Determines critical flow ratios ($y = v_c / s$), sums critical ratios for the intersection ($Y = \sum y$), applies Webster's formula $C_0 = \frac{1.5L + 5}{Y}$ to compute optimal cycle lengths, and allocates effective green time proportionally to phase demand ratios.
- **Outputs:** Recommended cycle length, phase-by-phase green split distributions, and arterial coordination offset values.
- **Safety Constraints:** Hardcodes statutory minimum pedestrian walk times and clearance intervals, rejecting or overriding any split calculation that breaches pedestrian safety minimums.
- **Optimization Flow:** Ingest critical volumes $\rightarrow$ compute flow ratios $\rightarrow$ apply Webster formula $\rightarrow$ constrain within min/max bounds $\rightarrow$ run safety validator check $\rightarrow$ output phase allocation arrays.
- **Module Boundaries:** The optimization engine executes strict deterministic Python mathematical calculations; the LLM is completely isolated from numerical computation to prevent hallucinations.

CHAPTER 10 — SIMULATION ENGINE

- **Current Scenario:** Evaluates baseline static timing plans against historical peak volume demands, yielding baseline control delay, queue length, and LOS grades.

- **Optimized Scenario:** Evaluates Webster-optimized green split allocations against the same volume demand, computing improved control delay, shorter queue lengths, and enhanced LOS grades.
- **Metrics:** Average control delay (sec/veh), peak queue length (meters), intersection LOS grade, estimated fuel savings (gallons/day), and CO_2 emission reductions (kg/day).
- **Assumptions:** Stable, steady-state arrival patterns; standard saturation flow rate of 1900 veh/hr/lane ; absence of heavy vehicle or steep grade impedance unless customized.
- **Limitations:** M/M/1 and Webster queuing models assume steady-state arrivals and can under-predict delay during extreme oversaturated conditions ($V/C > 1.2$) where queues compound exponentially.
- **Outputs:** Side-by-side comparative metric arrays, percentage reduction badges, and dynamic stress-test response values.

CHAPTER 11 — AI ARCHITECTURE

- **Prompt Generation:** Wraps structured JSON payloads of simulation before/after metrics into expert traffic engineering prompt templates that instruct Azure OpenAI to explain cause-and-effect drivers.
- **Structured JSON:** Enforces strict JSON data schemas passed between the deterministic Python calculation engine and the Azure OpenAI endpoint.
- **Explanation Generation:** Produces 2 to 3 paragraph plain-language Markdown narratives explaining why specific green time adjustments were made and confirming that pedestrian safety minimums were preserved.
- **Safety:** Strictly prohibits the LLM from executing mathematical calculations, altering numerical split values, or overriding safety constraints.
- **Fallback Behavior:** If Azure OpenAI times out or encounters rate limits, the system falls back to displaying a clean, deterministic template-based engineering summary.
- **Rate Limits:** Implements exponential backoff retry logic and client-side request throttling to comply with Azure OpenAI token limits.
- **Failure Handling:** Gracefully catches API errors, logs exceptions internally, and displays a non-blocking notification banner on the UI dashboard.

CHAPTER 12 — SECURITY

- **Input Validation:** Strictly validates all uploaded CSV files against required schema definitions, rejecting unparseable files or malicious payloads.
- **File Validation:** Enforces MIME-type checking (`text/csv`) and file size limits ($\leq 25 \text{ MB}$) on all user uploads.
- **Error Handling:** Implements centralized exception management to prevent raw stack traces and internal system details from leaking to client responses.
- **Secrets Management:** Stores database credentials, API keys, and Azure OpenAI tokens securely in environment variables via `.env` files, omitting secrets from source control.
- **Responsible AI:** Hardcodes mathematical safety limits into deterministic Python code to ensure AI-generated narratives cannot influence critical safety constraints.

- **Data Privacy:** Ensures municipal data isolation through Role-Based Access Control (RBAC), encrypting data in transit via TLS 1.3 and at rest via AES-256.

CHAPTER 13 — DEPLOYMENT

- **Local Development:** Run using a local Python virtual environment (`venv`), executing FastAPI via Uvicorn (`uvicorn backend.main:app --reload`) and the frontend via Streamlit (`streamlit run frontend/app.py`).
- **Hackathon Deployment:** Containerized multi-service deployment using Docker Compose, hosting the frontend, backend API, and analytical worker services on a cloud virtual private server (e.g., AWS EC2 or Azure VM).
- **Dependencies:** Python 3.10+, FastAPI, Streamlit, Pandas, NumPy, SciPy, Uvicorn, Requests, Pydantic, ReportLab.
- **Environment Variables:** `AZURE_OPENAI_API_KEY`, `AZURE_OPENAI_ENDPOINT`, `DATABASE_URL`, `JWT_SECRET_KEY`, `LOG_LEVEL`.
- **Docker (optional):** Multi-stage `Dockerfile` and `docker-compose.yml` orchestrating container networking and persistent volume storage for user project datasets.

CHAPTER 14 — TEAM DEVELOPMENT PLAN

Team Role Division (5 Members)

- **Member 1 (Frontend Lead):** Streamlit multi-page UI layout, GIS map integration, interactive sliders, and component styling.
- **Member 2 (Backend & API Lead):** FastAPI routing, database schema migrations, CSV upload handling, and report export suites.
- **Member 3 (Analytics & Optimization Lead):** Data cleaning algorithms, peak congestion clustering, V/C ratio calculation, and Webster optimization engine.
- **Member 4 (Simulation & Safety Lead):** M/M/1 queuing math implementation, before/after delay calculations, and statutory pedestrian safety validator.
- **Member 5 (AI Integration & Testing Lead):** Azure OpenAI prompt engineering, structured JSON payload formatting, unit testing suites, and hackathon presentation prep.

Git Workflow & Branch Strategy

- **Branch Strategy:** Gitflow branch model utilizing a stable `main` branch, a development integration `dev` branch, and feature-specific branches (`feature/analytics`, `feature/simulation`, etc.).
- **Merge Strategy:** Mandatory pull request (PR) reviews requiring at least one peer approval, passing automated unit test pipelines, and zero merge conflicts before merging into `dev` or `main`.

CHAPTER 15 — IMPLEMENTATION ROADMAP

Sprint 1: Foundation & Data Ingestion (Days 1–2)

- **Tasks:** Project repository setup, folder structure creation, database schema migrations, and CSV upload/validation backend endpoints.
- **Dependencies:** None.
- **Deliverables:** Functional CSV upload portal with automated error logging and data cleaning.
- **Success Criteria:** Valid CSV files parse successfully; invalid formats trigger descriptive error messages.

Sprint 2: Analytics & Optimization Engine (Days 3–4)

- **Tasks:** Implement rolling 1-hour peak window detection, V/C ratio calculations, Webster optimization math, and safety guardrail validators.
- **Dependencies:** Sprint 1 data ingestion schema.
- **Deliverables:** Core backend analytics and Webster green-split calculation modules.
- **Success Criteria:** Intersection V/C ratios correctly flag bottleneck nodes ($V/C > 1.0$), and safety validators successfully enforce pedestrian minimum walk times.

Sprint 3: Simulation & AI Integration (Days 5–6)

- **Tasks:** Build M/M/1 queuing simulation models, before/after metric comparisons, volume-surge stress test sliders, and Azure OpenAI narrative generation.
- **Dependencies:** Sprint 2 optimization engine outputs.
- **Deliverables:** Fully integrated simulation dashboard and AI narrative panel.
- **Success Criteria:** Simulation outputs display accurate delay reductions, and Azure OpenAI returns coherent plain-language summaries.

Sprint 4: Dashboard Polishing & Export (Day 7)

- **Tasks:** Finalize Streamlit dashboard UI styling, build NEMA CSV and PDF executive report generators, run end-to-end integration tests, and prepare final hackathon demo.
- **Dependencies:** Sprint 3 simulation and AI outputs.
- **Deliverables:** Production-ready hackathon MVP application with export suites.
- **Success Criteria:** Complete end-to-end corridor retiming workflow executes in under 5 minutes without errors.

CHAPTER 16 — TESTING STRATEGY

- **Unit Tests:** Test individual Python functions for data cleaning, timestamp parsing, Webster cycle length calculations, and safety boundary constraints using `pytest`.
- **Integration Tests:** Verify end-to-end API communication from CSV upload through analytics, optimization, simulation, and report export.

- **Analytics Validation:** Benchmark calculated V/C ratios and peak congestion windows against known Highway Capacity Manual sample problem datasets.
- **Simulation Validation:** Ensure M/M/1 queuing delay and queue length outputs maintain a variance within $\pm 10\%$ of standard traffic engineering formulas.
- **UI Testing:** Perform manual user journey walkthroughs across Streamlit screens to verify responsive layouts, tooltips, and safety lock animations.
- **Performance Testing:** Measure API response times under simulated concurrent user loads to ensure corridor optimization completes in under 5 seconds.

CHAPTER 17 — RISK ANALYSIS

- **Technical Risks:** Queuing models may under-predict delay under extreme oversaturated conditions ($V/C > 1.2$).
 - *Mitigation:* Implement prominent UI disclaimer labels stating models assume stable flow and actual field results will vary.
- **Integration Risks:** Azure OpenAI API timeouts or rate-limit throttling during live hackathon presentations.
 - *Mitigation:* Build robust fallback template text generation in case of API connection failure.
- **Performance Risks:** Large 7-day CSV files causing UI latency during parsing.
 - *Mitigation:* Offload heavy data processing to backend Pandas workers with asynchronous status polling.
- **Data Risks:** Corrupted or poorly formatted municipal CSV uploads crashing the parser.
 - *Mitigation:* Implement rigorous regex validation and automated imputation rules during data ingestion.
- **Business Risks:** Municipal engineers rejecting AI tools due to "black-box" skepticism.
 - *Mitigation:* Frame the platform strictly as an "Engineering Advisory Co-pilot" and provide transparent mathematical citations for every output.

FINAL IMPLEMENTATION CHECKLIST

- [] **Project Setup:** Initialize repository, establish branch protections, and configure `.env` files.
- [] **Folder Creation:** Build clean directory structure across frontend, backend, analytics, optimization, simulation, and AI modules.
- [] **Dependencies:** Install and lock Python requirements (`FastAPI`, `Streamlit`, `Pandas`, `NumPy`, `SciPy`, `ReportLab`).
- [] **Backend:** Implement database models, connection strings, and CRUD operations for projects, counts, and metadata.
- [] **Frontend:** Construct multi-page Streamlit dashboard, navigation bars, drag-and-drop upload zones, and comparative metrics cards.
- [] **Analytics:** Build CSV parser, anomaly cleaner, rolling 1-hour peak detector, and V/C ratio calculator.
- [] **Optimization:** Implement Webster cycle length algorithm, phase green split distribution, and coordination offsets.

- [] **Simulation:** Program M/M/1 queuing delay/queue estimators, before/after metric comparators, and volume-surge stress-test sliders.
- [] **AI:** Configure Azure OpenAI client, structured JSON payload formatters, and plain-language prompt templates.
- [] **Testing:** Execute unit and integration test suites covering data validation and safety boundary enforcement.
- [] **Deployment:** Containerize application via Docker Compose and verify local and cloud server execution.
- [] **Presentation:** Prepare 5-minute hackathon demo script showcasing CSV upload, bottleneck heatmaps, optimization sliders, AI narratives, and PDF exports.